

SudokuSolver

User Manual — Manuel utilisateur

Version 1.1.1

Bilingual Sudoku solver, generator and tutor

Solves any valid grid in milliseconds, generates puzzles by difficulty,
and explains each step the way a human would.

Solveur, générateur et tuteur de Sudoku bilingue

Résout n'importe quelle grille valide en quelques millisecondes, génère
des grilles par niveau de difficulté, et explique chaque étape
de la même façon qu'un humain.

This manual is split into two independent parts:

Part I — English • Part II — Français

Ce manuel est divisé en deux parties indépendantes :

Partie I — English • Partie II — Français

Contents

I	English	2
1	Overview	2
2	Installation	2
2.1	Requirements	2
2.2	Windows	2
2.3	Linux and macOS	2
2.4	Manual install	3
3	The interface	3
3.1	Main window	3
3.2	Keyboard	3
3.3	Cell states	3
4	Features	4
4.1	Generating a puzzle	4
4.2	Hint	4
4.3	Next step	4
4.4	Solve	4
4.5	Check	4
4.6	Candidates	4
4.7	Reset	4
4.8	File menu	5
4.9	Language menu	5
4.10	Help menu	5
5	Under the hood	5
5.1	Solver architecture	5
5.2	Human-style techniques	5
5.3	Generator	5
5.4	Threading and anti-freeze	6
5.5	Crash reports	6
6	Troubleshooting	6
6.1	“Python not found” on Windows	6
6.2	“Installation failed” after a Python upgrade	6
6.3	The generator is slow on Expert	6
6.4	I imported a puzzle and Solve says “no solution”	6
6.5	Where are my settings stored?	6
6.6	The app crashed — what should I do?	6
7	FAQ	7
8	Credits	7
II	Français	8
9	Présentation	8

10 Installation	8
10.1 Prérequis	8
10.2 Windows	8
10.3 Linux et macOS	8
10.4 Installation manuelle	9
11 L’interface	9
11.1 Fenêtre principale	9
11.2 Clavier	9
11.3 États d’une case	9
12 Fonctionnalités	10
12.1 Générer une grille	10
12.2 Indice	10
12.3 Étape suivante	10
12.4 Résoudre	10
12.5 Vérifier	10
12.6 Candidats	10
12.7 Réinitialiser	10
12.8 Menu Fichier	11
12.9 Menu Langue	11
12.10 Menu Aide	11
13 Sous le capot	11
13.1 Architecture du solveur	11
13.2 Techniques humaines	11
13.3 Générateur	12
13.4 Threads et anti-gel	12
13.5 Rapports de crash	12
14 Dépannage	12
14.1 “Python not found” sous Windows	12
14.2 “Installation failed” après une mise à jour de Python	12
14.3 Le générateur est lent en Expert	12
14.4 J’ai importé une grille et Résoudre dit “pas de solution”	12
14.5 Où sont stockés mes réglages ?	13
14.6 L’application a planté — que faire ?	13
15 FAQ	13
16 Crédits	13

Part I — English

1. Overview

SudokuSolver is a desktop application that solves, generates and teaches Sudoku puzzles. It is written in Python with a PyQt6 graphical interface and runs on Windows, Linux and macOS. The user interface is fully bilingual (French / English) and every widget has a tooltip in both languages.

Key features

- Fast solver with bitmask constraint propagation and MRV backtracking
- 8 human-style techniques with step-by-step explanations
- Puzzle generator with 4 calibrated difficulty levels
- Interactive 9×9 grid, pencil marks, real-time conflict highlighting
- Cross-platform (Windows, Linux, macOS)
- Crash reports anonymised automatically (no usernames, no paths)

2. Installation

2.1 Requirements

- Python 3.10 or newer
- `PyQt6` $\geq$ 6.5, `Pillow` $\geq$ 10.0
- Around 80 MB of free disk space for the virtual environment

2.2 Windows

The easiest way is to run `launch.bat` from the project folder. It detects Python, creates a local virtual environment in `%APPDATA%\SudokuSolver\venv`, installs dependencies on first launch, then starts the application silently (no console window).

```
launch.bat
```

If something goes wrong (corrupted venv, broken install, missing package), run the repair mode which wipes the venv and reinstalls everything cleanly:

```
launch.bat --repair
```

Tip

The launcher searches for Python in several standard locations (`%LOCALAPPDATA%\Programs\Python`, `C:\Python3xx`, `%ProgramFiles%`), then tries the Windows Python Launcher (`py -3`), then `python3` / `python` from the `PATH`. It deliberately avoids the Windows Store alias that would trigger an installation instead of running Python.

2.3 Linux and macOS

```
chmod +x launch.sh
./launch.sh
```

The virtual environment is created in `$XDG_DATA_HOME/SudokuSolver/venv` (or `~/.local/share/SudokuSolver` as a fallback).

2.4 Manual install

```
python -m venv venv
# Windows
venv\Scripts\activate
# Linux / macOS
source venv/bin/activate

pip install -r requirements.txt
python main.py
```

Warning

The project folder is portable across PCs (NAS-friendly): application data stays in the project folder, but the virtual environment is always local to each PC. Never copy the `venv` folder between machines — delete it and let the launcher recreate it.

3. The interface

3.1 Main window

The window has four main areas:

1. **Top bar** — difficulty selector and “New puzzle” button.
2. **Grid** — the interactive 9×9 board.
3. **Action bar** — Hint, Next step, Solve, Check, Candidates, Reset.
4. **Explanation label** and **status bar** below.

3.2 Keyboard

Key	Action
Arrow keys	Move selection
1–9	Enter digit in the current cell
0, Delete, Backspace	Clear cell
Mouse click	Select a cell

3.3 Cell states

- **Given** — original clue, cannot be edited, displayed in bold.
- **User entry** — digit you typed, displayed in a distinct colour.
- **Conflict** — cell contents violate a row, column or box rule; background turns red.
- **Candidates** — small digits showing possible values, toggled by the “Candidates” button.

4. Features

4.1 Generating a puzzle

Pick a difficulty in the combo box and click **New puzzle**. The generator creates a grid with rotational symmetry, guaranteed unique solution, and calibrates the clue count and hardest technique required to match the target difficulty.

Level	Clues	Hardest technique
Easy	36–45	Hidden Single
Medium	30–35	Hidden Pair
Hard	26–32	Box / Line Reduction
Expert	22–28	X-Wing

Generation runs in a background thread so the interface never freezes, even if the generator needs to retry several times to hit the target difficulty.

4.2 Hint

Click **Hint** to see the next logical move. The engine picks the simplest applicable technique, highlights the pattern cells in green and the eliminated candidates in red, and explains *why* the move works.

Tip

If no technique applies, the engine falls back to a guess based on the backtracking solver. This is very rare for generated puzzles but can happen on imported puzzles that require advanced techniques not yet implemented.

4.3 Next step

Same as Hint, but *applies* the step automatically. Chain several “Next step” clicks to watch the puzzle solve itself technique by technique.

4.4 Solve

Fills in the complete solution instantly. The fast solver runs in a background thread and is capped at 2 000 000 search nodes, which guarantees no freeze even on pathologically hard input.

4.5 Check

Compares every user-entered digit against the stored solution. Wrong cells are highlighted in red, correct cells in green. Useful when you’re not sure whether a deduction was valid.

4.6 Candidates

Toggles the pencil-mark display. When on, every empty cell shows its current candidate set in small digits. When off, the cells stay clean — useful for a less cluttered view when you’re confident about a region.

4.7 Reset

Clears all user entries and restores the puzzle to its generated state. The original clues are preserved.

4.8 File menu

- **New** — generate a new puzzle at the current difficulty.
- **Open** — load an 81-character puzzle string from a `.txt` or `.sdk` file. File size is capped at 10 KB.
- **Save** — save the current grid as an 81-character string (dots for empty cells).
- **Quit** — close the application.

4.9 Language menu

Switches the entire UI between French and English at runtime. The default is auto-detected from your system locale.

4.10 Help menu

- **How to play** — a short in-app guide.
- **Solving techniques** — a full bilingual catalogue of the 8 implemented techniques. See also the separate `guide_techniques.pdf` shipped with the project.
- **About** — version and credits.

5. Under the hood

5.1 Solver architecture

The solver represents candidate sets as 9-bit masks ($1 \ll (d-1)$ for digit d). Every cell knows the bitwise intersection of the allowed digits for its row, column and box. Placing a digit updates the peers' candidate masks in constant time.

Search uses a Minimum Remaining Values (MRV) heuristic: pick the empty cell with the fewest candidates, try each candidate in turn, propagate, recurse. This is the same pattern used by Peter Norvig's well-known solver, extended with an explicit node cap that bounds worst-case effort.

5.2 Human-style techniques

The human solver implements eight techniques, tried in order of increasing difficulty:

Technique	What it finds
Naked Single	Cell with only one candidate left
Hidden Single	Digit that fits in only one cell of a unit
Naked Pair	Two cells sharing exactly two candidates
Naked Triple	Three cells sharing three candidates
Hidden Pair	Two digits that fit in only the same two cells
Pointing Pair	Box candidates aligned on a single row or column
Box/Line Reduction	Line candidates confined to one box
X-Wing	2×2 rectangular elimination pattern

See `guide_techniques.pdf` for worked examples of every technique.

5.3 Generator

Generation starts from a random valid solution (shuffled backtracking), then removes clues symmetrically while verifying after each removal that the puzzle still has exactly one solution. The result is rated by counting clues and running the human solver to identify the hardest technique required. If the rating doesn't match the target difficulty, generation retries up to 40 times before falling back to the closest match.

5.4 Threading and anti-freeze

All long-running work (generation, solving) runs in dedicated `QThread` workers. The UI thread stays responsive and the window never freezes, even if you click “New puzzle” several times in a row or import a pathologically hard grid.

Worker lifecycle is handled defensively: when a thread finishes, its Python wrapper is cleared before Qt’s native object is deleted, so subsequent operations never touch a dangling reference.

5.5 Crash reports

Uncaught exceptions are caught by a `sys.excepthook` handler that writes a JSON crash report into the project’s `crash_reports/` folder. The report contains the traceback, Python version, OS and application version — *never* the username or home directory. Paths are rewritten to `~` and the username is replaced with `<user>`.

6. Troubleshooting

6.1 “Python not found” on Windows

The launcher couldn’t find a working Python interpreter. Install Python 3.10 or newer from <https://www.python.org/downloads/> and **make sure to tick “Add Python to PATH”** during installation. Then re-run `launch.bat`.

6.2 “Installation failed” after a Python upgrade

If you upgraded Python and the old venv is stale, run:

```
launch.bat --repair
```

This deletes the virtual environment and reinstalls everything cleanly.

6.3 The generator is slow on Expert

Expert puzzles that specifically require X-Wing are rare, so the generator may retry up to 40 times before falling back to a Hard-level puzzle with a slightly more constrained clue count. This is by design, and the background thread keeps the UI responsive throughout.

6.4 I imported a puzzle and Solve says “no solution”

Either the imported grid has a typo (wrong digit in a given), it already contains a conflict (check the red highlights), or it genuinely has no solution. The solver also bails out after 2 000 000 search nodes — this almost never triggers on valid 9×9 grids.

6.5 Where are my settings stored?

Everything stays inside the project folder (`crash_reports/`, `logs/`, puzzles history). The only thing outside is the virtual environment, which lives in `%APPDATA%\SudokuSolver\venv` on Windows or `$XDG_DATA_HOME/SudokuSolver/venv` on Linux and macOS for cross-PC portability.

6.6 The app crashed — what should I do?

Check the `crash_reports/` folder inside the project. The most recent JSON file contains an anonymised traceback you can attach to a GitHub issue. Usernames and home paths are replaced automatically, so it is safe to share.

7. FAQ

Can I play on a phone?

No — this is a desktop application. For mobile, use any Sudoku app from your platform’s store.

Does it phone home?

No. No network requests, no analytics, no telemetry. Crash reports are written locally and only shared if you choose to.

What Sudoku formats are supported?

81-character strings with digits 1-9 and any of ., 0, - or space for empty cells. Line breaks are tolerated.

Why is the “Solving techniques” guide a separate PDF?

Because it is a reference you want to browse while solving. The main manual (this document) is the “how to use the app” guide; the techniques guide is the “how to solve Sudoku” reference.

8. Credits

SudokuSolver is MIT-licensed. Thanks to Peter Norvig for the classic constraint-propagation solver design, and to the Sudoku solving community for decades of accumulated technique catalogues.

Issues and contributions welcome at:
<https://github.com/ARP273-ROSE/SudokuSolver>

Part II — Français

9. Présentation

SudokuSolver est une application de bureau qui résout, génère et enseigne les grilles de Sudoku. Écrite en Python avec une interface graphique PyQt6, elle fonctionne sous Windows, Linux et macOS. L'interface est entièrement bilingue (français / anglais) et chaque contrôle dispose d'une infobulle dans les deux langues.

Fonctionnalités clés

- Solveur rapide : propagation de contraintes par masques binaires et backtracking MRV
- 8 techniques humaines avec explications pas-à-pas
- Générateur 4 niveaux, calibré par nombre d'indices et technique maximale requise
- Grille interactive 9×9, candidats, détection d'erreurs en temps réel
- Multiplateforme (Windows, Linux, macOS)
- Rapports de crash automatiquement anonymisés (pas de nom d'utilisateur ni de chemin)

10. Installation

10.1 Prérequis

- Python 3.10 ou plus récent
- `PyQt6` $\geq$ 6.5, `Pillow` $\geq$ 10.0
- Environ 80 Mo d'espace disque pour l'environnement virtuel

10.2 Windows

Le plus simple est de lancer `launch.bat` depuis le dossier du projet. Le script détecte Python, crée un environnement virtuel local dans `%APPDATA%\SudokuSolver\venv`, installe les dépendances au premier lancement, puis démarre l'application silencieusement (pas de fenêtre console).

```
launch.bat
```

Si quelque chose se passe mal (venv corrompu, installation cassée, paquet manquant), utilisez le mode réparation qui supprime le venv et réinstalle tout proprement :

```
launch.bat --repair
```

Astuce

Le launcher cherche Python dans plusieurs emplacements standard (`%LOCALAPPDATA%\Programs\Python`, `C:\Python3xx`, `%ProgramFiles%`), puis essaie le Windows Python Launcher (`py -3`), puis `python3` / `python` dans le `PATH`. Il évite délibérément l'alias Windows Store qui déclencherait une installation au lieu de lancer Python.

10.3 Linux et macOS

```
chmod +x launch.sh
./launch.sh
```

L'environnement virtuel est créé dans `$XDG_DATA_HOME/SudokuSolver/venv` (ou `~/.local/share/SudokuSolver/venv` par défaut).

10.4 Installation manuelle

```
python -m venv venv
# Windows
venv\Scripts\activate
# Linux / macOS
source venv/bin/activate

pip install -r requirements.txt
python main.py
```

Attention

Le dossier du projet est portable entre PC (compatible NAS) : les données applicatives restent dans le projet, mais le `venv` est toujours local à chaque machine. Ne jamais copier le dossier `venv` entre machines — supprimez-le et laissez le launcher le recréer.

11. L'interface

11.1 Fenêtre principale

La fenêtre comporte quatre zones principales :

1. **Barre supérieure** — sélecteur de difficulté et bouton “Nouvelle grille”.
2. **Grille** — le plateau 9×9 interactif.
3. **Barre d'actions** — Indice, Étape suivante, Résoudre, Vérifier, Candidats, Réinitialiser.
4. **Zone d'explication** et **barre d'état** en bas.

11.2 Clavier

Touche	Action
Flèches	Déplacer la sélection
1–9	Saisir un chiffre dans la case courante
0, Suppr, Retour arr.	Effacer la case
Clic souris	Sélectionner une case

11.3 États d'une case

- **Donnée** — indice original, non modifiable, affiché en gras.
- **Saisie utilisateur** — chiffre que vous avez tapé, dans une couleur distincte.
- **Conflit** — le contenu enfreint une règle ligne, colonne ou bloc ; le fond devient rouge.
- **Candidats** — petits chiffres montrant les valeurs possibles, activés par le bouton “Candidats”.

12. Fonctionnalités

12.1 Générer une grille

Choisissez une difficulté dans la liste déroulante et cliquez sur **Nouvelle grille**. Le générateur crée une grille avec symétrie rotationnelle, solution unique garantie, et calibre le nombre d’indices et la technique la plus difficile requise pour correspondre à la difficulté cible.

Niveau	Indices	Technique maximale
Facile	36–45	Singleton caché
Moyen	30–35	Paire cachée
Difficile	26–32	Réduction bloc / ligne
Expert	22–28	X-Wing

La génération s’exécute dans un thread d’arrière-plan, l’interface ne gèle jamais, même si le générateur doit réessayer plusieurs fois pour atteindre la difficulté cible.

12.2 Indice

Cliquez sur **Indice** pour voir le prochain coup logique. Le moteur choisit la technique applicable la plus simple, met en évidence les cases du motif en vert et les candidats éliminés en rouge, puis explique *pourquoi* le coup fonctionne.

Astuce

Si aucune technique ne s’applique, le moteur bascule sur une hypothèse basée sur le solveur par backtracking. C’est très rare pour les grilles générées, mais peut arriver sur des grilles importées nécessitant des techniques avancées non encore implémentées.

12.3 Étape suivante

Comme Indice, mais *applique* l’étape automatiquement. Enchaînez plusieurs clics “Étape suivante” pour regarder la grille se résoudre technique par technique.

12.4 Résoudre

Remplit la solution complète instantanément. Le solveur rapide s’exécute dans un thread d’arrière-plan et est plafonné à 2 000 000 de nœuds de recherche, ce qui garantit l’absence de gel même sur des entrées pathologiques.

12.5 Vérifier

Compare chaque chiffre saisi par l’utilisateur à la solution stockée. Les cases incorrectes sont surlignées en rouge, les cases correctes en vert. Utile quand vous n’êtes pas sûr qu’une déduction était valide.

12.6 Candidats

Active ou désactive l’affichage des candidats. Activé, chaque case vide affiche son ensemble courant de candidats en petits chiffres. Désactivé, les cases restent propres — utile pour une vue moins chargée quand vous êtes confiant sur une région.

12.7 Réinitialiser

Efface toutes les saisies utilisateur et restaure la grille dans son état généré. Les indices originaux sont préservés.

12.8 Menu Fichier

- **Nouvelle** — génère une nouvelle grille à la difficulté courante.
- **Ouvrir** — charge une chaîne de 81 caractères depuis un fichier `.txt` ou `.sdk`. Taille du fichier plafonnée à 10 Ko.
- **Enregistrer** — sauvegarde la grille courante sous forme de chaîne de 81 caractères (points pour les cases vides).
- **Quitter** — ferme l'application.

12.9 Menu Langue

Permet de basculer toute l'interface entre français et anglais à chaud. Par défaut, la langue est détectée automatiquement depuis la locale système.

12.10 Menu Aide

- **Comment jouer** — un court guide intégré.
- **Techniques de résolution** — un catalogue bilingue complet des 8 techniques implémentées. Voir aussi le fichier `guide_techniques.pdf` fourni avec le projet.
- **À propos** — version et crédits.

13. Sous le capot

13.1 Architecture du solveur

Le solveur représente les ensembles de candidats sous forme de masques 9 bits ($1 \ll (d-1)$ pour le chiffre d). Chaque case connaît l'intersection bit à bit des chiffres autorisés pour sa ligne, sa colonne et son bloc. Placer un chiffre met à jour les masques de candidats des pairs en temps constant.

La recherche utilise l'heuristique MRV (*Minimum Remaining Values*) : choisir la case vide avec le moins de candidats, essayer chaque candidat, propager, récurser. C'est le schéma utilisé par le célèbre solveur de Peter Norvig, étendu avec un plafond explicite de nœuds qui borne l'effort dans le pire cas.

13.2 Techniques humaines

Le solveur humain implémente huit techniques, essayées par ordre de difficulté croissante :

Technique	Ce qu'elle détecte
Singleton nu	Case n'ayant plus qu'un candidat
Singleton caché	Chiffre ne pouvant tenir que dans une case d'une unité
Paire nue	Deux cases partageant exactement deux candidats
Triplet nu	Trois cases partageant trois candidats
Paire cachée	Deux chiffres ne pouvant tenir que dans les deux mêmes cases
Paire pointante	Candidats d'un bloc alignés sur une même ligne ou colonne
Réduction bloc/ligne	Candidats d'une ligne confinés à un seul bloc
X-Wing	Motif d'élimination rectangulaire 2×2

Voir `guide_techniques.pdf` pour des exemples détaillés de chaque technique.

13.3 Générateur

La génération part d’une solution valide aléatoire (backtracking mélangé), puis retire les indices de façon symétrique en vérifiant après chaque retrait que la grille conserve exactement une solution. Le résultat est évalué en comptant les indices et en faisant tourner le solveur humain pour identifier la technique la plus difficile requise. Si l’évaluation ne correspond pas à la difficulté cible, la génération réessaie jusqu’à 40 fois avant de se rabattre sur la correspondance la plus proche.

13.4 Threads et anti-gel

Tout le travail long (génération, résolution) s’exécute dans des `QThread` dédiés. Le thread de l’interface reste réactif et la fenêtre ne gèle jamais, même si vous cliquez plusieurs fois d’affilée sur “Nouvelle grille” ou importez une grille pathologiquement difficile.

Le cycle de vie des workers est géré de façon défensive : quand un thread se termine, son wrapper Python est vidé *avant* que l’objet natif Qt ne soit détruit, de sorte que les opérations suivantes ne touchent jamais une référence pendante.

13.5 Rapports de crash

Les exceptions non rattrapées sont capturées par un gestionnaire `sys.excepthook` qui écrit un rapport JSON dans le dossier `crash_reports/` du projet. Le rapport contient la *traceback*, la version de Python, l’OS et la version de l’application — *jamais* le nom d’utilisateur ni le dossier personnel. Les chemins sont réécrits en `~` et le nom d’utilisateur est remplacé par `<user>`.

14. Dépannage

14.1 “Python not found” sous Windows

Le launcher n’a pas trouvé d’interpréteur Python fonctionnel. Installez Python 3.10 ou plus récent depuis <https://www.python.org/downloads/> et **cochez bien “Add Python to PATH”** pendant l’installation. Puis relancez `launch.bat`.

14.2 “Installation failed” après une mise à jour de Python

Si vous avez mis à jour Python et que l’ancien venv est obsolète, exécutez :

```
launch.bat --repair
```

Cela supprime l’environnement virtuel et réinstalle tout proprement.

14.3 Le générateur est lent en Expert

Les grilles Expert qui nécessitent spécifiquement un X-Wing sont rares, donc le générateur peut réessayer jusqu’à 40 fois avant de se rabattre sur une grille Difficile avec un nombre d’indices légèrement plus contraint. C’est intentionnel, et le thread d’arrière-plan garde l’interface réactive pendant toute l’opération.

14.4 J’ai importé une grille et Résoudre dit “pas de solution”

Soit la grille importée contient une erreur de saisie (mauvais chiffre dans un indice), soit elle contient déjà un conflit (vérifiez les cases rouges), soit elle n’a réellement pas de solution. Le solveur abandonne aussi après 2 000 000 de nœuds de recherche — cela ne se déclenche presque jamais sur une grille 9×9 valide.

14.5 Où sont stockés mes réglages ?

Tout reste dans le dossier du projet (`crash_reports/`, `logs/`, historique des grilles). La seule chose à l'extérieur est l'environnement virtuel, qui se trouve dans `%APPDATA%\SudokuSolver\venv` sous Windows ou `$XDG_DATA_HOME/SudokuSolver/venv` sous Linux et macOS, pour permettre la portabilité entre PC.

14.6 L'application a planté — que faire ?

Vérifiez le dossier `crash_reports/` dans le projet. Le fichier JSON le plus récent contient une *traceback* anonymisée que vous pouvez joindre à un ticket GitHub. Les noms d'utilisateur et les chemins personnels sont remplacés automatiquement, c'est donc sans risque de partager.

15. FAQ

Puis-je jouer sur téléphone ?

Non — c'est une application de bureau. Pour le mobile, utilisez une application Sudoku dédiée sur le store de votre plateforme.

Y a-t-il de la télémétrie ?

Aucune. Aucune requête réseau, aucune analytique, aucune télémétrie. Les rapports de crash restent locaux sauf si vous choisissez de les partager.

Quels formats de Sudoku sont pris en charge ?

Les chaînes de 81 caractères avec chiffres 1-9 et l'un de `.`, `0`, `-` ou espace pour les cases vides. Les retours à la ligne sont tolérés.

Pourquoi le guide des techniques est-il un PDF séparé ?

Parce que c'est une référence que vous voudrez consulter pendant la résolution. Le manuel principal (ce document) est le guide "comment utiliser l'application" ; le guide des techniques est la référence "comment résoudre un Sudoku".

16. Crédits

SudokuSolver est distribué sous licence MIT. Merci à Peter Norvig pour le schéma classique du solveur par propagation de contraintes, et à la communauté Sudoku pour des décennies de catalogue de techniques accumulées.

Les tickets et contributions sont les bienvenus sur :
<https://github.com/ARP273-ROSE/SudokuSolver>